

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I KANMANI K B (192524306) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Problem

There are **3 doctors**:

- D1
- D2
- D3

and **3 shifts**:

- Morning (M)
- Afternoon (A)
- Night (N)

We need to assign exactly one doctor to each shift while satisfying all constraints.

Constraints

1. **D1 cannot work Night**
 - $D1 \neq N$
2. **D2 must work before D3**
 - $\text{Shift}(D2) < \text{Shift}(D3)$
 - Morning < Afternoon < Night
3. **Only one doctor per shift**
4. **D3 cannot work Morning**
 - $D3 \neq M$
5. **Every doctor gets exactly one shift**

Step 1: Define the State

We can represent an assignment as:

[Morning, Afternoon, Night]

For example:

[D1, D2, D3]

means:

- Morning $\rightarrow$ D1
- Afternoon $\rightarrow$ D2
- Night $\rightarrow$ D3

Backtracking assigns doctors one by one and checks the constraints after every assignment.

Step 2: Start with an Empty Assignment

[M, A, N]

[_, _, _]

No doctor has been assigned yet.

Step 3: Try D1 for Morning

Assign:

Morning $\rightarrow$ D1

Current state:

[D1, _, _]

Constraint checks

- D1 cannot Night $\rightarrow$ D1 is Morning $\rightarrow$ **Satisfied**
- One doctor per shift $\rightarrow$ **Satisfied**
- D2 before D3 $\rightarrow$ Cannot check yet
- D3 cannot Morning $\rightarrow$ No D3 in Morning $\rightarrow$ **Satisfied**

Therefore, continue.

Step 4: Try D2 for Afternoon

Assign:

Afternoon $\rightarrow$ D2

Current state:

[D1, D2, _]

Constraint checks

- D1 is not Night $\rightarrow$ **Satisfied**
- D2 is Afternoon.
- D3 must be after D2.
- The only remaining shift after Afternoon is Night.
- Therefore D3 must be Night.

Continue.

Step 5: Assign D3 to Night

Current state:

[D1, D2, D3]

Check all constraints

Constraint	Check	Result
D1 cannot Night	D1 = Morning	✓

D2 before D3	Afternoon < Night	✓
One doctor per shift	All different	✓
D3 cannot Morning	D3 = Night	✓
Every doctor assigned once	D1, D2, D3 each once	✓

All constraints are satisfied.

Final Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

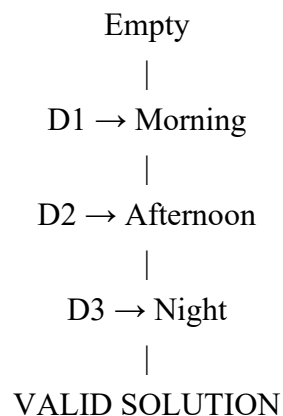
Final Answer

D1 → Morning

D2 → Afternoon

D3 → Night

Complete Backtracking Search Tree

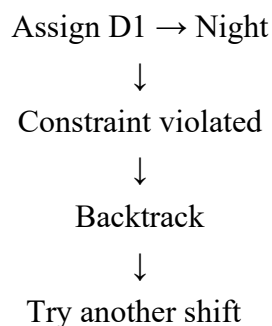


Since the first attempted path satisfies all constraints, **no backtracking is actually required after reaching the solution.**

Why is it called Backtracking?

If an assignment had violated a constraint, we would undo the last assignment and try another possibility.

For example:



2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Given Conditions

Grid:

5×5

Movement:

- Up
- Down
- Left
- Right

Cost of each movement: 1

Obstacles cannot be crossed.

Manhattan Distance:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

where:

- (x_n, y_n) = current position
- (x_G, y_G) = goal position

Assumed Grid

Since the question does not provide the actual 5×5 grid/figure, we need to use a sample obstacle arrangement to demonstrate the algorithm.

BFS Search

BFS explores nodes level by level.

Initial queue:

[S]

The cost of the start node is:

$$g(S) = 0$$

Step 1 – Expand S

Position:

(1,1)

Possible movements:

- Up → outside grid ✗
- Left → outside grid ✗
- Down → obstacle ✗
- Right → (1,2) ✓

Queue becomes:

[(1,2)]

Cost:

$g(1,2) = 1$

Manhattan distance:

$$h = |1 - 5| + |2 - 5|$$

$$h = 4 + 3 = 7$$

Step 2 – Expand (1,2)

Possible movements:

- Up → outside ✗
- Left → (1,1) already visited
- Right → obstacle (1,3) ✗
- Down → (2,2) ✓

Queue:

[(2,2)]

Cost:

$g = 2$

Manhattan:

$$h = |2 - 5| + |2 - 5| = 6$$

Step 3 – Expand (2,2)

Possible movements:

- Up → (1,2) visited
- Down → (3,2) ✓

- Left $\rightarrow$ obstacle (2,1) ✗
- Right $\rightarrow$ obstacle (2,3) ✗

Queue:

[(3,2)]

Cost:

g = 3

Manhattan:

$$h = |3 - 5| + |2 - 5| = 5$$

Step 4 – Expand (3,2)

Possible movements:

- Up $\rightarrow$ (2,2) visited
- Left $\rightarrow$ (3,1) ✓
- Right $\rightarrow$ (3,3) ✓
- Down $\rightarrow$ (4,2) obstacle ✗

Queue:

[(3,1), (3,3)]

Both have:

g = 4

Manhattan distances:

For (3,1):

$$h = |3 - 5| + |1 - 5| = 6$$

For (3,3):

$$h = |3 - 5| + |3 - 5| = 4$$

Step 5 – Expand (3,1)

Possible movements:

- Up $\rightarrow$ (2,1) obstacle
- Down $\rightarrow$ (4,1) ✓
- Right $\rightarrow$ (3,2) visited
- Left $\rightarrow$ outside

Queue:

[(3,3), (4,1)]

Step 6 – Expand (3,3)

Possible movements:

- Up $\rightarrow$ (2,3) obstacle
- Down $\rightarrow$ (4,3) obstacle
- Left $\rightarrow$ (3,2) visited
- Right $\rightarrow$ (3,4) ✓

Queue:

[(4,1), (3,4)]

Therefore:

S $\rightarrow$ (1,2) $\rightarrow$ (2,2) $\rightarrow$ (3,2)
 $\rightarrow$ (3,3) $\rightarrow$ (3,4) $\rightarrow$ (3,5)
 $\rightarrow$ (4,5) $\rightarrow$ G

Total Cost

There are 8 movements.

Since each movement costs 1:

$$Cost = 8 \times 1$$

$$\boxed{Cost = 8}$$

Important conclusion

Because all movements have equal cost:

BFS finds the shortest path and therefore the optimal path.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Problem

An autonomous rescue robot operates inside a disaster-affected building.

The robot must:

- Start from S
- Find a survivor at G
- Avoid obstacles
- Avoid dangerous/risky areas
- Minimize travel cost
- Work with incomplete information
- Update its knowledge while moving

This is a more complex AI problem because the robot **does not know the complete environment in advance**.

3(a). Analysis of Constraints

Constraint 1: Movement

The robot can move:

UP

DOWN

LEFT

RIGHT

It cannot move diagonally.

Therefore, each state has at most four possible successor states.

Constraint 2: Obstacles

Some cells are blocked.

For example:

S . # .

. # . .

. . . G

represents an obstacle.

The robot cannot enter an obstacle cell.

Therefore, when generating successor states:

IF cell is obstacle

Do not add cell to search

Constraint 3: Limited Sensing

The robot can only observe nearby cells.

It does not have complete information about the building.

For example, from:

 ?
 |
? -- R -- ?
 |
 ?

the robot can only observe adjacent cells.

It does not know what is beyond the visible region.

This makes the environment:

Partially Observable

The robot must maintain an internal representation of what it has discovered.

Constraint 4: Different Movement Costs

Normal movement:

Cost = 1

Risky zone:

Movement cost = $1 + 2 = 3$

Therefore:

Cell Type	Cost
Normal	1
Risky	3
Obstacle	∞ / Not allowed

This is very important.

The robot should not simply find the path with the smallest number of movements.

It should find the path with the **lowest total cost**.

Example

Consider:

$S \rightarrow A \rightarrow B \rightarrow G$

All normal:

$1 + 1 + 1 = 3$

Another path:

$S \rightarrow C \rightarrow D \rightarrow G$

Suppose C is risky:

$3 + 1 + 1 = 5$

Although both paths contain three movements, the first path is cheaper.

Therefore, the robot should choose:

$S \rightarrow A \rightarrow B \rightarrow G$

Constraint 5: Dynamic Knowledge

Because the robot has limited sensing, it may discover new information.

Initially:

Robot knows:

$S \rightarrow ? \rightarrow ? \rightarrow G$

After moving:

Robot discovers obstacle

The robot updates its map:

$S \rightarrow A \rightarrow \# \rightarrow G$

It then searches for another route.

This is called:

Online Search

The robot does not plan everything once and blindly follow it.

Instead:

Observe

↓

Update knowledge

↓

Plan

↓

Move

↓

Observe again

↓

Update

↓

Re-plan

3(b). Solution Approach

Suitable Strategy

A combination of:

Uniform Cost Search + Online Search Agent

is appropriate.

Why?

- Movement costs are not all equal.
- Risky cells cost more.
- The robot has incomplete information.
- The robot must update its knowledge.
- UCS finds the minimum-cost path based on currently known information.
- Online search allows the robot to react to newly discovered obstacles.

Step-by-Step Approach

Step 1: Start at S

The robot starts at:

S

Initial path cost:

$$g(S) = 0$$

Step 2: Sense the Environment

The robot checks adjacent cells.

For example:

```
  ?
  |
? ---- S ---- .
  |
  #
```

The robot discovers:

- One free cell
- One obstacle
- Other cells unknown

Step 3: Generate Available Actions

The robot considers:

UP

DOWN

LEFT

RIGHT

It removes:

- Obstacles

- Already invalid positions

Step 4: Calculate Cost

For every available cell:

Normal cell

$$Cost = CurrentCost + 1$$

Risky cell

$$Cost = CurrentCost + 3$$

The robot adds these states to the priority queue.

UCS Priority Queue

The priority queue is sorted by:

$$g(n)$$

where $g(n)$ is the cumulative cost from the starting position.

Example:

Priority Queue

Node A $\rightarrow$ cost 2

Node B $\rightarrow$ cost 3

Node C $\rightarrow$ cost 5

Node D $\rightarrow$ cost 7

The robot always expands:

Node A

because it has the lowest cost.

Step 5: Expand Lowest-Cost State

Suppose:

$S \rightarrow A$

Cost:

1

From A, suppose:

$A \rightarrow B = 1$

$A \rightarrow C = 3$

Then:

B cumulative cost = 2

C cumulative cost = 4

Priority queue:

[B, 2]

[C, 4]

B is expanded first.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided